Core Templates — High-Level Reference

The single most generic, abstract template for each question type — the reusable skeleton you adapt per problem. Worked solutions, quick-reference tables, and the full problem index live in `template.md` / each category file.

Conventions, Skeleton & Core API

Every template file in this folder follows the same two-part structure:

1. **Template algorithm + template code** — the reusable pattern, with specific LeetCode problems used as concrete examples inside the code.
2. **Quick reference table + question/solution** — a table listing every problem (with a *Variation from Template* column), followed by each problem's full solution where deviations are marked `// ← VARIATION: explanation`.

Generic Test Harness

Use this boilerplate to run any single problem locally. Replace `data` / `solve()` with the problem's input and logic. Work through the six steps in the comment before coding.

```java
class Question {

    // ── PROBLEM-SOLVING PROCESS ──────────────────────────────────────
    // 1. INPUT / OUTPUT  : types, sizes, ranges; what to return; examples.
    // 2. HIGH-LEVEL ALGO : pick the pattern (two-ptr / BFS / DP / ...); state the idea in 1 line.
    // 3. IMPLEMENTATION  : data structures, loop/recursion, invariants.
    // 4. EDGE CASES      : empty, size 0/1, duplicates, negatives, overflow, all-same, not-found.
    // 5. COMPLEXITY      : time & space; is it within limits?
    // 6. OPTIMIZATION    : reduce time/space (hashing, two-pointer, rolling array, pruning, early exit
    // ─────────────────────────────────────────────────────────────────

    private int data;

    public Question(int data) {
        this.data = data;
    }

    public void solve() {
        // TODO
    }
}

public class Solution {

    public static void main(String[] args) {
        Question q = new Question(0);
        q.solve();
    }
}
```

Naming Conventions (enforced across all template files)

Context	Convention
Index variables	`i`, `j`, `k` (never `left`, `right`, `low`, `mid`, `high`)
Linked list	`sentinel`, `previous`, `current`
Queue	`Queue<Integer> queue = new ArrayDeque<>();`
BFS over tree	`Queue<TreeNode> queue = new ArrayDeque<>();`
Grid directions	`int[] dr = {1,-1,0,0}, dc = {0,0,1,-1};` (DOWN, UP, RIGHT, LEFT); iterate `for (int dir=0; dir<4; dir++)` using `i+dr[dir]`, `j+dc[dir]`
Character frequency	`count[ch - 'a']++;`
Digit frequency	`count[ch - '0']++;`
Char to integer build	`num = num * 10 + (ch - '0');`

Common Java API Cheat-Sheet

The APIs used over and over across these templates. Grouped by what you reach for.

Initialize the core structures

```
Map<Integer, Integer> map   = new HashMap<>();      // key → value
Map<Integer, List<Integer>> adj = new HashMap<>();  // adjacency / buckets
Set<Integer> set            = new HashSet<>();       // membership
List<Integer> list          = new ArrayList<>();     // dynamic array

Deque<Integer> stack = new ArrayDeque<>();           // STACK: push / pop / peek
Deque<Integer> queue = new ArrayDeque<>();           // QUEUE: offer / poll / peek
                                                     // (ArrayDeque beats java.util.Stack / LinkedList)
PriorityQueue<Integer> minHeap = new PriorityQueue<>();                         // min-heap (default)
PriorityQueue<Integer> maxHeap = new PriorityQueue<>(Collections.reverseOrder()); // max-heap
PriorityQueue<int[]>   pq      = new PriorityQueue<>((a, b) -> a[1] - b[1]);  // by field, min-first

TreeMap<Integer, Integer> tmap = new TreeMap<>();    // sorted keys: floor/ceiling/first/last
TreeSet<Integer> tset          = new TreeSet<>();    // sorted set: floor/ceiling/higher/lower
LinkedHashSet<Integer> lset    = new LinkedHashSet<>(); // insertion-ordered set (LFU buckets)
Random rand                    = new Random();       // rand.nextInt(n) → 0..n-1 (RandomizedSet)

int[]     arr  = new int[n];                         // defaults to 0
boolean[] seen = new boolean[n];                     // defaults to false
int[][]   grid = new int[m][n];                      // 2D, all 0
int[]     dr   = {1,-1,0,0}, dc = {0,0,1,-1};  // DOWN,UP,RIGHT,LEFT; iterate for(dir=0..3) i+dr[dir],
```

Map / Set / List operations

```
// ── Map ──
map.put(k, v);
map.getOrDefault(k, 0);                                  // safe read with default
map.merge(k, 1, Integer::sum);                           // ← counting frequency (most common)
map.computeIfAbsent(k, x -> new ArrayList<>()).add(v); // ← get-or-create a bucket, then mutate
map.putIfAbsent(k, v);                                   // ← keep FIRST occurrence (e.g. value→earliest in
//   putIfAbsent(k, v)        : value is EAGER (built up front); inserts only if k absent;
//                              RETURNS the OLD value (null if was absent). Use to keep first write.
//   computeIfAbsent(k, f)    : value is LAZY (f runs only if k absent); inserts & RETURNS the
//                              current value (existing or just-made) → chain `.add(...)` on it.
//                              Idiom for adjacency lists / buckets — no allocation when present.
map.containsKey(k);  map.remove(k);
for (Map.Entry<Integer,Integer> e : map.entrySet()) { e.getKey(); e.getValue(); e.setValue(v); }
map.keySet();  map.values();

// ── Set ──
set.add(x);  set.contains(x);  set.remove(x);

// ── List ──
list.add(x);  list.get(i);  list.set(i, x);
list.remove(list.size() - 1);                            // ← pop in backtracking
list.size();  list.isEmpty();  list.addAll(other);
Collections.sort(list);                                  // ascending
list.sort((a, b) -> b - a);                              // custom / descending
Collections.reverse(list);                               // ← reverse path after building it backwards
```

Stack / Queue / Heap (all via the same two ops)

```
// Deque as STACK (LIFO)               // Deque as QUEUE (FIFO)
stack.push(x);                          queue.offer(x);
int top = stack.pop();                  int head = queue.poll();
int look = stack.peek();                int look = queue.peek();
stack.isEmpty();                        queue.isEmpty();

// Deque BOTH ENDS — needed for the MONOTONE DEQUE (sliding-window max/min)
// and for addFirst-style level building (zigzag BFS).
deque.offerLast(x);   deque.pollLast();   deque.peekLast();    // back  (push/pop/peek tail)
deque.offerFirst(x);  deque.pollFirst();  deque.peekFirst();   // front (push/pop/peek head)
deque.addLast(x);     deque.addFirst(x);                       // same as offer*, throw if capacity-bou

// PriorityQueue (heap)
pq.offer(x);  int best = pq.poll();  int look = pq.peek();  pq.size();
pq.remove(x); // ← O(n) removal of an arbitrary element (lazy-deletion alternative in #480)
```

Integer / Long limits & overflow-safe idioms

```
Integer.MAX_VALUE;   //  2147483647  (2^31 - 1)   — sentinel for "min so far"
Integer.MIN_VALUE;   // -2147483648  (-2^31)      — sentinel for "max so far"
Long.MAX_VALUE;      // when sums can overflow int, accumulate in long
int k = i + (j - i) / 2;                 // ← avoid (i + j) overflow in binary search
Integer.compare(a, b);                   // ← overflow-safe comparator; use instead of (a - b)
(a, b) -> Integer.compare(a[1], b[1]);// ← safe PriorityQueue/Arrays.sort comparator
Double.compare(a, b);                    // ← comparator for double[] heaps (probability, median)
```

Char ↔ int ↔ String conversions

```
ch - 'a';                           // 'a'..'z'  → 0..25   (lowercase bucket index)
ch - '0';                           // '0'..'9'  → 0..9    (digit value)
(char) ('a' + i);                   // 0..25 → 'a'..'z'
num = num * 10 + (ch - '0');        // ← build a number digit by digit

Integer.parseInt("123");            // String → int (throws on bad input)
Integer.valueOf(123);               // int → Integer (boxed)
String.valueOf(123);                // int/char/etc → String
Character.getNumericValue('7');     // char → 7

// char classification
Character.isDigit(ch);  Character.isLetter(ch);  Character.isLetterOrDigit(ch);
Character.isWhitespace(ch);  Character.toLowerCase(ch);  Character.toUpperCase(ch);
```

String & StringBuilder

```
s.length();  s.charAt(i);  s.substring(i, j);        // [i, j)
s.toCharArray();  s.split(" ");  s.equals(t);  s.compareTo(t);
s.indexOf("ab");  s.isEmpty();
s.startsWith("ab");  s.endsWith("z");  s.contains("x"); // prefix / suffix / substring tests
s.toLowerCase();  s.toUpperCase();                   // case-normalize (e.g. case-insensitive compare)
s.trim();  s.replace('a', 'b');  s.repeat(n);        // strip ends / swap chars / repeat

StringBuilder builder = new StringBuilder();
builder.append(x);  builder.reverse();  builder.toString();
builder.charAt(i);  builder.setCharAt(i, c);  builder.deleteCharAt(i);  builder.length();
String.join(",", list);                              // list → "a,b,c"
```

Arrays utilities

```
Arrays.sort(arr);                                    // ascending in place
Arrays.sort(intervals, (a, b) -> a[0] - b[0]);       // 2D by first field
Arrays.fill(dp, Integer.MAX_VALUE);                  // seed a dp/dist array
Arrays.equals(window, need);                         // ← element-wise array compare (anagram check)
Arrays.copyOf(arr, len);  Arrays.copyOfRange(arr, i, j);
System.arraycopy(src, srcPos, dst, dstPos, len);     // ← fast block copy (merge step)
Arrays.asList(1, 2, 3);                              // fixed-size List view
int sum = Arrays.stream(arr).sum();
int max = Arrays.stream(arr).max().getAsInt();

// List → array  (toArray makes REFERENCE arrays only; primitives need a stream)
String[]  s   = strList.toArray(new String[0]);   // List<T> → T[]    (typed, preferred)
Object[]  obj = list.toArray();                    // List<T> → Object[]
int[]  a = list.stream().mapToInt(Integer::intValue).toArray();   // List<Integer> → int[]
long[] b = list.stream().mapToLong(Long::longValue).toArray();    // List<Long>    → long[]
// array → List
List<Integer> l = Arrays.stream(a).boxed().collect(Collectors.toList());  // int[] → List<Integer> (or
List<String>  t = Arrays.asList(strArr);                                   // T[]   → fixed-size List<T
```

From → To	API
`List<T>` → `T[]`	`list.toArray(new T[0])`
`List<T>` → `Object[]`	`list.toArray()`
`List<Integer>` → `int[]`	`list.stream().mapToInt(Integer::intValue).toArray()`
`List<Long>` → `long[]`	`list.stream().mapToLong(Long::longValue).toArray()`
`int[]` → `List<Integer>`	`Arrays.stream(a).boxed().toList()`
`T[]` → `List<T>`	`Arrays.asList(arr)`

Math & bit operations

```
Math.max(a, b);  Math.min(a, b);  Math.abs(x);
Math.pow(a, b);  Math.sqrt(x);  Math.ceil(x);  Math.floor(x);

n & (n - 1);           // clear lowest set bit
n & (-n);              // isolate lowest set bit
1 << i;                // bit i set  (use 1L << i for i ≥ 31)
(n >> i) & 1;          // read bit i
(n & 1) == 0;          // ← even   (last bit 0)
(n & 1) == 1;          // ← odd    (last bit 1)
n >> 1;                // ← divide by 2 (drop last bit);  n << 1 = multiply by 2
Integer.bitCount(n);  // # of set bits
Integer.toBinaryString(n);
```

Optimization idioms

Common speed/space wins used throughout the templates.

```
// —— NEGATIVE-SAFE MODULO —— replaces the manual ((x % n) + n) % n
Math.floorMod(x, n);                  // always in [0, n-1] even when x < 0 (prefix-sum % k)

// —— int[] INSTEAD OF HashMap for a bounded key space ——
int[] count = new int[26];            // O(1) no-boxing freq vs HashMap<Character,Integer>
count[ch - 'a']++;                    // also faster to compare: Arrays.equals(a, b)

// —— 1D ROLLING ARRAY for DP —— drop a dimension when row i only needs row i-1
int[] dp = new int[n + 1];            // O(n) space instead of O(m·n)
for (...) for (int j = target; j >= w; j--) dp[j] += dp[j - w];  // 0/1 knapsack: iterate j DOWN

// —— FIXED-SIZE HEAP for top-k —— O(n log k) beats sorting O(n log n)
if (heap.size() > k) heap.poll();     // keep only k best; min-heap for "k largest"

// —— LAZY DELETION over pq.remove(obj) —— pq.remove is O(n); skip stale entries instead
if (entry[0] != dist[node]) continue; // Dijkstra: ignore outdated heap entries

// —— ArrayDeque over Stack / LinkedList —— faster, no synchronization, both ends O(1)
Deque<Integer> stack = new ArrayDeque<>();   // never use java.util.Stack

// —— StringBuilder over String += in a loop —— O(n) vs O(n²)
StringBuilder builder = new StringBuilder();      // append, then builder.toString() once

// —— EARLY EXIT / PRUNING —— return the moment the answer is decided
if (found) return result;             // e.g. backtracking bounds
```

Template Index

File	Category	Core Pattern
`binary_search.md`	Binary Search	Closed `[i,j]`, half-open, minimize/maximize answer space
`two_pointers.md`	Two Pointers	Opposite ends, same-direction write pointer, Dutch flag
`prefix_sum_hashmap.md`	Prefix Sum	Count/length with HashMap, tree path sums
`sorting.md`	Sorting	Merge sort, quick sort, quick select, count-during-merge
`sliding_window.md`	Sliding Window	Fixed, max-variable, min-variable windows
`linked_list.md`	Linked List	Delete/filter, reversal, fast/slow, merge, composite
`dfs.md`	DFS / Backtracking	Tree return-up/pass-down, grid DFS, graph 3-color, backtracking, BST
`bfs.md`	BFS	Level-order with size snapshot, position indexing
`graph.md`	Graph	BFS shortest path, topo sort, union find, Dijkstra, bipartite, Prim's MST
`dynamic_programming.md`	Dynamic Programming	1D/2D/grid/interval DP, knapsack, state machine
`stack_queue.md`	Stack / Queue / Heap	Monotone stack/deque, two heaps, priority queue
`greedy.md`	Greedy	Sort-by-end, sort-by-start, sort + heap, non-interval
`string.md`	String	Frequency counting, hashing, bucket sort, expand-from-center
`bit_manipulation.md`	Bit Manipulation	XOR cancel, mod-k state machine, bitmask as set, power checks
`design.md`	Design	HashMap + linked list (LRU), frequency maps (LFU), array + map
`math.md`	Math	Fast power, base conversion, sieve, digit manipulation

---

Two Pointers `two_pointers.md`

Two patterns — pick based on whether pointers converge or march together.

Pattern 1 — Opposite Two Pointers

Variables: `i` = left pointer · `j` = right pointer · scan range `[i, j]` closed **Pseudocode:**

```
set i to the left end, j to the right end
while the pointers haven't crossed:
  if the current pair hits the target:
    record it, then step both ends inward
  else if the value is too small:
    move the left end right to grow it
  else (too large):
    move the right end left to shrink it
```

```
// MENTAL MODEL: two ends close in on the answer; each comparison rules out one whole end of the range.
// WHY sorted: moving inward only safely eliminates half the space if the array is sorted
int i = 0, j = n - 1;
while (i < j) {
    if (condition == target) {
        /* record */ i++; j--;
    } else if (tooSmall) {
        i++;
    } else {
        j--;
    }
}
```

Sort first. `i` and `j` converge inward. Used when the array is sorted and you can eliminate half the search space each step.

Sliding Window `sliding_window.md`

Three variants — pick by what the problem asks for.
`i` = left boundary (inclusive), `j` = right boundary (loop variable, inclusive).
Window = `[i, j]`, size = `j - i + 1`.

The Three Templates

Variables: `i` = left edge (inclusive) · `j` = right edge / loop variable (inclusive), window = `[i, j]`, size = `j - i + 1` · `n` = array length · `k` = target window size · `result` = answer (max length, or min length seeded to MAX) **Pseudocode:**

```
FIXED window of size k:
    i = 0
    for j = 0..n-1:
        add nums[j] to window state
        if window size (j - i + 1) == k:
            record the window (it is exactly size k)
            remove nums[i] from state; i = i + 1

MAX variable window (longest valid):
    i = 0; result = 0
    for j = 0..n-1:
        add nums[j] to state
        while window violates the constraint:
            remove nums[i] from state; i = i + 1
        result = max(result, j - i + 1)   (window is valid here)

MIN variable window (shortest valid):
    i = 0; result = +infinity
    for j = 0..n-1:
        add nums[j] to state
        while window is valid:
            result = min(result, j - i + 1)   (record while still valid)
            remove nums[i] from state; i = i + 1   (then shrink to try smaller)
```

```
// FIXED window of size k — shrink exactly once when full
// a constant-width window slides one step at a time; add the new cell, drop the old one.  — WHEN: "sub
int i = 0;
for (int j = 0; j < n; j++) {
    // 1. add nums[j] to state
    if (j - i + 1 == k) {
        // 2. record (window is exactly size k)
        // 3. remove nums[i] from state
        i++;
    }
}

// MAX variable window — find LONGEST valid window
// grow greedily; shrink only enough to restore validity, then the window is the best ending here.  — W
int i = 0, result = 0;
for (int j = 0; j < n; j++) {
    // 1. add nums[j] to state
    while (/* window exceeds constraint */) {  // shrink WHILE window exceeds constraint
        // remove nums[i] from state
        i++;
    }
    result = Math.max(result, j - i + 1);   // record after window is valid
}

// MIN variable window — find SHORTEST valid window
// grow until valid, then shrink aggressively while still valid to find the tightest fit.  — WHEN: "sho
int i = 0, result = Integer.MAX_VALUE;
for (int j = 0; j < n; j++) {
    // 1. add nums[j] to state
    while (/* window VALID */) {                  // record then shrink WHILE window still valid
        result = Math.min(result, j - i + 1);  // record while valid
        // remove nums[i] from state
        i++;                                    // then try to shrink
    }
}
```

Frequency Map Trick (used by 567 and 76)

Both use a `need[]` array and a `required` counter to avoid scanning the map each step.

Variables: `need[c]` = how many more of char `c` the window still needs (positive = still short, zero/negative = surplus) · `required` = total chars still missing; hits 0 when the window covers all of t · `i` = left edge, `j` = right edge
Pseudocode:

```
Expanding (add char at j):
    if need[char] was still positive (> 0): required = required - 1   (filled one more)
    need[char] = need[char] - 1

Shrinking (remove char at i):
    if need[char] was already 0 or surplus (>= 0): required = required + 1  (now short one)
    need[char] = need[char] + 1
    i = i + 1
```

```
// Expanding j — add s.charAt(j):
if (need[s.charAt(j)]-- > 0) required--;   // was needed → satisfied one more

// Shrinking i — remove s.charAt(i):
if (need[s.charAt(i)]++ >= 0) required++;  // was 0 (used up) → now short one
i++;
```

`need[c]` is positive when still needed, zero or negative when excess.
`required` reaches 0 exactly when all characters of t are covered.

Prefix Sum + HashMap `prefix_sum_hashmap.md`

Core idea: `subarray sum [l, r] = prefix[r+1] - prefix[l]`.
To find a subarray with a target property, rephrase as: "has a prefix with value X been seen before?"
Store that X in a HashMap for O(1) lookup.

Core Template

Variables: `map` = prefixSum→count or index seen so far · `sum` = running prefix sum · `lookup` = the complement prefix we search for · `i` = current scan position **Pseudocode:**

```
create empty map of prefix → (count or index)
seed map with prefix 0 (count=1 for counting, index=-1 for length)
sum = 0
for each index i:
    add nums[i] to sum
    lookup = transform(sum)        // sum-k, sum%k, etc.
    use map.get(lookup) to update result
    store sum (or sum%k) into map with count or index i
```

```
// a subarray is the difference of two prefixes; remember prefixes seen so far and look up the compleme
Map<Long, Integer> map = new HashMap<>();
map.put(0L, ???);    // seed: empty prefix (sum=0) seen before index 0
long sum = 0;
// result = 0 (count) or Integer.MAX_VALUE (length sentinel) depending on problem

for (int i = 0; i < nums.length; i++) {
    sum += nums[i];
    long lookup = transform(sum);          // what to look up (sum-k, sum%k, etc.)
    // use map.get(lookup) → update result
    map.put(storeKey(sum), storeValue(i)); // what to store (sum or sum%k → count or index)
}
```

What changes per problem:

Variable	Count problems (560, 974)	Length problems (325, 523)
Map value	`count` (how many times seen)	`first index` (earliest occurrence)
Seed value	`map.put(0L, 1)`	`map.put(0L, -1)`
On lookup hit	`count += map.get(lookup)`	`maxLen = max(i - map.get(lookup))`
Overwrite map?	Yes (merge count)	No (keep first index only)

Binary Search `binary_search.md`

Binary search appears in two forms: searching an **array index**, or searching an **answer value**.

The Only Template — `[i, j)`, find first `k` with `condition(k)`

There is one binary search. Define a monotone `condition(k)` (`false…false TRUE…true`); the loop returns the **first `k` where `condition(k)` is true**. Then look at `i`: if `i == n` no index qualified, otherwise `i` is the boundary — check the value to decide.

Variables: `[i, j)` = half-open search range · `k` = midpoint · `condition(k)` = monotone test (false…false→TRUE…true) · **answer = `i`**, always in `[0, length]` (`i == length` ⇒ nothing satisfied it). **Range:** index search → `[0, length)`. **Answer-space** (searching a value, not an index) → `[min, max + 1)` (half-open, so the largest value `max` is reachable). **Pseudocode:**

```
i = 0, j = length        # half-open [i, j) ; for answer-space use [min, max+1)
while range non-empty (i < j):
    k = midpoint of i and j
    if condition(k) is false: answer is to the right, set i = k+1
    else: k might be the answer, set j = k
```

```java
int i = 0, j = nums.length;       // [i, j)
// find first k that meets condition(k)
while (i < j) {
    int k = i + (j - i) / 2;
    if (!condition(k)) {          // condition false → answer is strictly to the right
        i = k + 1;
    } else {                      // condition true  → k might be the answer
        j = k;
    }
}
// i in [0, nums.length]; if i == nums.length, no k met condition(k)
```

Pick `condition(k)`:

Want	`condition(k)`	Answer after loop
`lowerBound` — first `>= target`	`nums[k] >= target`	`i`
`upperBound` — first `> target`	`nums[k] > target`	`i`
exact search (#704)	`nums[k] >= target`	`i < n && nums[i] == target ? i : -1`
insert position (#35)	`nums[k] >= target`	`i`
first / last occurrence (#34)	`>=` then `>`	`lowerBound`, `upperBound - 1`
count of target	—	`upperBound - lowerBound`
minimize feasible X	`feasible(k)` over value range	`i`
maximize feasible X	`!feasible(k)` (first infeasible)	`i - 1`

The rule: the answer is always `i`. Guard `i == nums.length` (nothing met the condition) before reading `nums[i]`, then confirm the value.

How to Choose

Signal	`condition(k)`
Sorted array, find exact value	`arr[k] >= target`, then check `arr[i] == target`
Insert position / first occurrence	`arr[k] >= target` → return `i` (= `lowerBound`)
Last occurrence / count	`arr[k] > target` (= `upperBound`); last = `upperBound − 1`, count = `upper − lower`
Rotated / peak slope search	`condition(k)` compares `arr[k]` to a neighbor or endpoint
"minimum X such that feasible(X)"	`condition(k) = feasible(k)` over value range → return `i`
"maximum X such that feasible(X)"	`condition(k) = !feasible(k)` over `[lo, hi+1)` → return `i − 1`

The one rule: define `condition(k)` so it is monotone `false…false TRUE…true`, then the answer is the first `k` that flips it true (`j = k` on true, `i = k + 1` on false). For MAXIMIZE, make the condition "first infeasible" and subtract 1 — same loop, no ceiling mid.

Sorting `sorting.md`

All recursive sorting uses **half-open intervals `[start, end)`** throughout.
Inside merge and partition: `i`, `j`, `k` as scan/write pointers (consistent with binary search convention).

Template 1 — Merge Sort

Variables: `[start, end)` half-open range being sorted · `mid` = split point · `temp` = scratch buffer · `i` = left scan pointer `[start,mid)` · `j` = right scan pointer `[mid,end)` · `k` = write pointer into temp **Pseudocode:**

```
mergeSort(nums, start, end):
    if range has 0 or 1 elements: return        // already sorted
    mid = midpoint of [start, end)
    mergeSort left half  [start, mid)
    mergeSort right half [mid, end)
    merge the two sorted halves

merge:
    i = start, j = mid, k = start
    while either half has elements left:
        pick smaller front of the two halves (left wins ties → stable)
        write it to temp[k], advance that half's pointer and k
    copy temp back into nums[start, end)
```

```java
// split in half until trivially sorted, then merge two sorted halves into one.  — WHEN: need stable so
// Entry point
public void mergeSort(int[] nums) {
    mergeSort(nums, 0, nums.length, new int[nums.length]);
}

// [start, end) half-open
private void mergeSort(int[] nums, int start, int end, int[] temp) {
    if (end - start <= 1) return;         // base case: end - start <= 1 → 0 or 1 elements, already sort
    int mid = start + (end - start) / 2;
    mergeSort(nums, start, mid, temp);   // sort [start, mid)
    mergeSort(nums, mid,   end, temp);   // sort [mid, end)
    merge(nums, start, mid, end, temp);
}

// i scans left [start,mid), j scans right [mid,end), k writes to temp
private void merge(int[] nums, int start, int mid, int end, int[] temp) {
    int i = start, j = mid, k = start;
    while (i < mid || j < end) {
        if (j >= end || (i < mid && nums[i] <= nums[j])) {
            temp[k++] = nums[i++];
        } else {
            temp[k++] = nums[j++];
        }
    }
    for (i = start; i < end; i++) nums[i] = temp[i];
}
```

Template 2 — Quick Sort (3-way Dutch Flag Partition)

Variables: `[start, end)` half-open range · `pivot` = chosen partition value · `i` = boundary of `<pivot` region · `k` = scan/cursor pointer · `j` = boundary of `>pivot` region (from the right) **Pseudocode:**

```
quickSort(nums, start, end):
    if range has 0 or 1 elements: return
    pivot = middle element's value
    i = start, k = start, j = end-1     // [start,i)<p | [i,k)==p | [k,j] unknown | (j,end)>p
    while k <= j:
        if nums[k] < pivot:  swap into < region, advance k and i
        elif nums[k] == pivot: advance k
        else:                swap to > region, shrink j (don't advance k)
    recurse on < region [start, i)
    recurse on > region [k, end)         // ==pivot middle is final
```

```java
// pick a pivot, partition into <pivot | ==pivot | >pivot, recurse on the two ends only.  — WHEN: in-pl
// Entry point — no extra array needed (in-place)
public void quickSort(int[] nums) {
    quickSort(nums, 0, nums.length);
}

// [start, end) half-open
private void quickSort(int[] nums, int start, int end) {
    if (end - start <= 1) return;
    int pivot = nums[start + (end - start) / 2];
    int i = start, k = start, j = end - 1;
    // Invariant: [start,i) < pivot  |  [i,k) == pivot  |  [k,j] unknown  |  (j,end) > pivot
    while (k <= j) {
        if (nums[k] < pivot) {
            swap(nums, k++, i++);
        } else if (nums[k] == pivot) {
            k++;
        } else {
            swap(nums, k, j--);
        }
    }
    // After: [start,i) < pivot, [i,k) == pivot, [k,end) > pivot
    quickSort(nums, start, i);   // recurse on < region
    quickSort(nums, k, end);     // recurse on > region
}

private void swap(int[] nums, int i, int j) {
    int t = nums[i]; nums[i] = nums[j]; nums[j] = t;
}
```

Template 3 — Quick Select (partial sort — k-th element)

Extends quick sort: after partition, only recurse into the side that contains `target` index.

Variables: `[start, end)` half-open range · `target` = 0-indexed position we want settled · `pivot` = partition value · `i` = boundary of `<pivot` region · `k` = scan/cursor pointer · `j` = boundary of `>pivot` region **Pseudocode:**

```
quickSelect(nums, start, end, target):
    pivot = middle element's value
    partition into [start,i)<p | [i,k)==p | [k,end)>p   // same 3-way as quick sort
    if target < i:        recurse into left  [start, i)
    elif target < k:      target sits in ==pivot region → return pivot
    else:                 recurse into right [k, end)
```

```java
// same partition as quick sort, but recurse into only the one side holding target → O(n) avg.  — WHEN:
// Returns value at 0-indexed position target after sorting
private int quickSelect(int[] nums, int start, int end, int target) {
    int pivot = nums[start + (end - start) / 2];
    int i = start, k = start, j = end - 1;
    while (k <= j) {
        if (nums[k] < pivot) {
            swap(nums, k++, i++);
        } else if (nums[k] == pivot) {
            k++;
        } else {
            swap(nums, k, j--);
        }
    }
    // [start,i) < pivot, [i,k) == pivot, [k,end) > pivot
    if (target < i) {
        return quickSelect(nums, start, i, target);
    } else if (target < k) {
        return pivot;                          // target lands in == region
    } else {
        return quickSelect(nums, k, end, target);
    }
}
```

Template 4 — Counting Sort

O(n + range). Use when values are bounded integers.

Variables: `min` / `max` = value range bounds · `buckets[v−min]` = count of value `v` · `i` = write pointer back into arr · `v` = current value index into buckets **Pseudocode:**

```
if array empty: return
find min and max values
buckets = array sized (max−min+1), all zero
for each x in arr: increment buckets[x−min]      // tally occurrences
i = 0
for v from 0 to last bucket:
    while bucket v still has count: write (v+min) into arr[i], advance i
```

```java
// tally how many times each value occurs, then emit values in order — no comparisons.  — WHEN: integer
public void countingSort(int[] arr) {
    if (arr.length == 0) return;
    int min = Arrays.stream(arr).min().getAsInt();
    int max = Arrays.stream(arr).max().getAsInt();
    int[] buckets = new int[max - min + 1];
    for (int x : arr) buckets[x - min]++;
    int i = 0;
    for (int v = 0; v < buckets.length; v++)
        while (buckets[v]-- > 0) arr[i++] = v + min;
}
```

Linked List `linked_list.md`

Four patterns cover almost every linked list problem.
Consistent naming throughout: `sentinel`, `previous`, `current`, `next`, `slow`, `fast`.

When to Use — Signal → Pattern

Signal in the prompt	Pattern
"remove / delete / skip" nodes by value or duplicate	Delete / Filter (sentinel + `previous`)
"reverse" the list, a sub-range, or in k-groups	Reversal
"middle node", "cycle", "nth node from the end"	Fast / Slow
"merge two sorted lists" (or k lists)	Merge (sentinel + heap for k)
chained steps ("reorder", "palindrome", "sort")	Composite (combine the above)

All Four Templates

Variables: `sentinel` = dummy node before head · `previous` = last kept node · `current` = node under inspection · `next` = saved successor · `slow` / `fast` = 1×/2× walkers · `a` / `b` = the two input lists **Pseudocode:**

```
make sentinel pointing at head; previous = sentinel, current = head
while current exists:
    if current should be deleted: rewire previous.next to skip current
    else: advance previous to current
    advance current
return sentinel.next
previous = null, current = head
while current exists:
    save next = current.next
    point current backward at previous
    advance previous to current, current to next
return previous as new head
slow = head, fast = head
while fast and fast.next exist:
    advance slow one step, fast two steps
make sentinel; current = sentinel
while both a and b exist:
    splice the smaller head onto current; advance that list
    advance current
attach whichever list remains
return sentinel.next
```

```java
// 1. DELETE / FILTER  — sentinel guards head removal; previous tracks last kept node
// previous always points at the last node you decided to keep, so re-wiring it skips anything unwanted
ListNode sentinel = new ListNode(0);
sentinel.next = head;
ListNode previous = sentinel, current = head;
while (current != null) {
    if (shouldDelete(current)) {
        previous.next = current.next;   // skip node
    } else {
        previous = current;             // keep node, advance previous
    }
    current = current.next;
}
return sentinel.next;

// 2. REVERSAL — re-wire one node at a time
// flip each arrow to point backward; previous is the growing reversed list trailing behind current.  —
ListNode previous = null, current = head;
while (current != null) {
    ListNode next = current.next;   // save before overwrite
    current.next = previous;        // reverse pointer
    previous = current;             // advance
    current = next;
}
return previous;  // new head

// 3. FAST / SLOW — two pointers at different speeds
// fast covers twice the distance, so when it hits the end slow is exactly halfway; in a loop they must
ListNode slow = head, fast = head;
while (fast != null && fast.next != null) {
    slow = slow.next;
    fast = fast.next.next;
}
// slow is at middle  (or cycle detection: slow == fast → cycle found)

// 4. MERGE — sentinel collects nodes from two lists in order
// repeatedly splice the smaller head onto a growing result tail, like a zipper.  — WHEN: "merge two (o
ListNode sentinel = new ListNode(0);
ListNode current = sentinel;
while (a != null && b != null) {
    if (a.val <= b.val) {
        current.next = a;
        a = a.next;
    } else {
        current.next = b;
        b = b.next;
    }
    current = current.next;
}
current.next = (a != null) ? a : b;
return sentinel.next;
```

`previous` stays on the last **kept** node. When deleting, re-wire `previous.next` to skip `current`. When keeping, advance `previous` to `current`. Always advance `current`.

String `string.md`

Core idioms first, then problems grouped by pattern.
For sliding window string problems (#3, #76, #567, #424), see `sliding_window.md`.

Core Idioms

Variables: `count` = frequency vector (index = char/digit bucket) · `ch - 'a'` = 0–25 bucket index · `ch - '0'` = 0–9 digit value · `num` = number built so far · `builder` = encoded/result string · `buckets` = lists indexed by frequency · `expand(i,j)` = palindrome length from a center **Pseudocode:**

```
idiom 1 — char count for letters:
  make count of size 26
  for each char: count[ch-'a'] += 1
idiom 2 — char count for digits:
  make count of size 10
  for each char: count[ch-'0'] += 1
idiom 3 — char to integer:
  num = 0
  for each char left to right: num = num*10 + (char - '0')
idiom 4 — anagram hash key (frequency-based):
  count the 26 letters of s
  build a string from each letter label followed by its count
  return it (same key for all anagrams)
  alternative: sort the chars and use the sorted string as key
idiom 5 — bucket sort by frequency:
  count all ASCII chars
  make buckets indexed by frequency
  for each char with count>0: add it to buckets[its count]
  walk buckets from highest frequency down, append each char that many times
idiom 6 — expand from center:
  while i in bounds and j in bounds and chars at i and j match: move i left, j right
  return j - i - 1 (length of palindrome found)
```

```java
// a lowercase string is a length-26 frequency vector; most string problems are vector ops on it.  — WH
// 1. CHAR COUNT — lowercase letters
int[] count = new int[26];
// WHY ch - 'a': ASCII 'a'=97, so 'a'→0, 'b'→1, ..., 'z'→25 — a clean 0–25 bucket index into count[].
for (char ch : s.toCharArray()) count[ch - 'a']++;     // ← ch - 'a' maps a→0, b→1, ..., z→25

// 2. CHAR COUNT — digits 0–9
int[] count = new int[10];
for (char ch : s.toCharArray()) count[ch - '0']++;     // ← ch - '0' maps '0'→0, '9'→9

// 3. CHAR TO INTEGER — build number digit by digit
int num = 0;
for (int i = 0; i < s.length(); i++)
    num = num * 10 + (s.charAt(i) - '0');              // ← shift left × 10, add new digit

// 4. ANAGRAM HASH KEY — frequency-based (bucket sort style) O(k) vs sort O(k log k)
// WHY it works: anagrams share identical letter frequencies, so encoding the frequency vector
//   ITSELF gives a unique key for the whole group — O(k) to build vs O(k log k) to sort the chars.
String hashKey(String s) {
    int[] count = new int[26];
    for (char ch : s.toCharArray()) count[ch - 'a']++;
    StringBuilder builder = new StringBuilder();
    for (int i = 0; i < 26; i++) {
        builder.append((char)('a' + i));                 // ← letter as label
        builder.append(count[i]);                        // ← its count
    }
    return builder.toString();                           // e.g., "a2b0c1...z0"
}
// Alternative: sort the characters (simpler, slightly slower)
char[] chars = s.toCharArray();
Arrays.sort(chars);
String key = new String(chars);                        // anagrams → same sorted key

// 5. BUCKET SORT — sort by frequency without comparison sort
int[] count = new int[128];                            // ASCII
for (char ch : s.toCharArray()) count[ch]++;
List<Character>[] buckets = new List[s.length() + 1]; // index = frequency
for (int i = 0; i < 128; i++) {
    if (count[i] > 0) {
        int freq = count[i];
        if (buckets[freq] == null) buckets[freq] = new ArrayList<>();
        buckets[freq].add((char) i);
    }
}
StringBuilder builder = new StringBuilder();
for (int freq = buckets.length - 1; freq > 0; freq--) {    // ← descending frequency
    if (buckets[freq] == null) continue;
    for (char ch : buckets[freq])
        for (int i = 0; i < freq; i++) builder.append(ch);
}

// 6. EXPAND FROM CENTER — palindrome check in O(1) space
int expand(String s, int i, int j) {          // i == j: odd length; i+1 == j: even length
    while (i >= 0 && j < s.length() && s.charAt(i) == s.charAt(j)) { i--; j++; }
    return j - i - 1;                         // ← length of palindrome found
}
```

Stack / Queue / Heap `stack_queue.md`

Four data structures — each with a canonical template and representative problems.

All Templates

Variables: `stack` = LIFO ArrayDeque (holds indices for monotone variants) · `queue` = monotone deque (front = window answer) · `minPQ` / `maxPQ` / `pq` = priority queue (heap) · `maxHeap` = lower half, `minHeap` = upper half for median
Pseudocode:

```
STACK: push/pop/peek all at the front (LIFO)
MONO-DECREASING (next greater): while current > stack top, pop and record current as its answer; push inc
MONO-INCREASING (next smaller / span): while current < stack top, pop and compute width to new top bounda
MONO-DEQUE (window max): drop back while smaller than current, push; drop front if out of window; front =
PRIORITY QUEUE: offer/poll/peek the min (or max with reverse comparator)
TWO HEAPS: push to maxHeap then shift its top to minHeap; rebalance so maxHeap >= minHeap; median from th
```

```java
// 1. STACK (LIFO) — use ArrayDeque, not Stack class
Deque<Integer> stack = new ArrayDeque<>();
stack.push(x);     // add to top
stack.pop();       // remove from top
stack.peek();      // view top, no remove

// 2A. MONOTONE DECREASING STACK — next GREATER element
// keep only candidates still waiting for a bigger neighbor; current resolves them all.  — WHEN: "next/
// Invariant: stack values decrease from bottom to top
// Pop when current is GREATER than top → top found its next greater
Deque<Integer> stack = new ArrayDeque<>();  // stores indices
for (int i = 0; i < n; i++) {
    while (!stack.isEmpty() && nums[stack.peek()] < nums[i])
        result[stack.pop()] = nums[i];     // nums[i] is next greater for popped index
    stack.push(i);
}
// Remaining in stack: no next greater element exists

// 2B. MONOTONE INCREASING STACK — next SMALLER element / span width
// each popped bar's reach extends until something shorter blocks it on both sides.  — WHEN: "largest r
// Invariant: stack values increase from bottom to top
// Pop when current is SMALLER than top → use popped index to compute width/span
Deque<Integer> stack = new ArrayDeque<>();  // stores indices
for (int i = 0; i < n; i++) {
    while (!stack.isEmpty() && nums[stack.peek()] > nums[i]) {
        int height = nums[stack.pop()];
        int width  = stack.isEmpty() ? i : i - stack.peek() - 1;  // span between boundaries
        // process height * width
    }
    stack.push(i);
}

// 3. MONOTONE DEQUE — sliding window max/min
// the front is the current window's answer; anything a newer-and-bigger value beats is dead weight.  —
// Deque stores INDICES; values at those indices are decreasing front-to-back (for max)
Deque<Integer> queue = new ArrayDeque<>();
for (int i = 0; i < n; i++) {
    while (!queue.isEmpty() && nums[queue.peekLast()] <= nums[i])
        queue.pollLast();               // remove smaller elements — they can never be window max
    queue.offerLast(i);
    if (queue.peekFirst() < i - k + 1) queue.pollFirst();  // evict out-of-window elements
    if (i >= k - 1) result[i - k + 1] = nums[queue.peekFirst()];
}

// 4. PRIORITY QUEUE
PriorityQueue<Integer> minPQ = new PriorityQueue<>();                      // min-heap (default)
PriorityQueue<Integer> maxPQ = new PriorityQueue<>(Collections.reverseOrder()); // max-heap
PriorityQueue<int[]>    pq   = new PriorityQueue<>((a, b) -> a[1] - b[1]); // custom: sort by index 1
pq.offer(x);   // add
pq.poll();     // remove and return min/max
pq.peek();     // view min/max without removing

// 5. TWO HEAPS — maintain median in O(log n)
// split the data at the median; the median is always sitting on the two heaps' tops.  — WHEN: "running
// maxHeap = lower half (smaller numbers), minHeap = upper half (larger numbers)
// Invariant: maxHeap.size() == minHeap.size() or maxHeap.size() == minHeap.size() + 1
PriorityQueue<Integer> maxHeap = new PriorityQueue<>(Collections.reverseOrder());
PriorityQueue<Integer> minHeap = new PriorityQueue<>();
void addNum(int num) {
    maxHeap.offer(num);
    minHeap.offer(maxHeap.poll());              // push one to minHeap (possibly num itself)
    if (maxHeap.size() < minHeap.size())
        maxHeap.offer(minHeap.poll());          // rebalance: maxHeap always ≥ minHeap in size
```

```
}
double findMedian() {
    return maxHeap.size() > minHeap.size()
        ? maxHeap.peek()
        : (maxHeap.peek() + minHeap.peek()) / 2.0;
}
```

BFS (Tree) `bfs.md`

Core structure: `Queue<TreeNode> queue = new ArrayDeque<>()`.
One template: snapshot the level size, then drain exactly that many nodes per level.

The Template — Level-Aware BFS

Variables: `queue` = nodes waiting to be processed · `size` = level snapshot (count of nodes in the current level) · `level` = current depth counter · `node` = node polled this step · `i` = index within the level **Pseudocode:**

```
make an empty queue
put root in the queue
level = 0
while the queue is not empty:
    size = how many nodes are in the queue right now (one whole level)
    level = level + 1
    repeat size times (index i = 0..size-1):
        node = take the front of the queue
        process node (level known; i==0 -> first, i==size-1 -> last)
        if node has a left child, add it to the queue
        if node has a right child, add it to the queue
```

```
// LEVEL-AWARE PROCESSING — snapshot size to make per-level decisions
// the queue holds exactly one full level; freeze its size, then drain just that many.  — WHEN: "level
Queue<TreeNode> queue = new ArrayDeque<>();
queue.offer(root);
int level = 0;
while (!queue.isEmpty()) {
    int size = queue.size();       // ← snapshot: all nodes currently in queue = this level
    level++;
    for (int i = 0; i < size; i++) {
        TreeNode node = queue.poll();
        // process node (know: level, i==0 → first, i==size-1 → last)
        if (node.left  != null) queue.offer(node.left);
        if (node.right != null) queue.offer(node.right);
    }
}
```

Always snapshot `size` first — it's how you know where each level ends. (If you truly don't need levels, just drop the `size` loop and poll node-by-node.)

When to Use — Signal → Pattern

Problem signal phrase	Pattern / group
"level order", "values per level", "each row"	Group 1 — collect entire level
"average / max / sum of each level"	Group 2 — aggregate per level
"right side view" (last in level), "bottom-left" (first in level)	Group 2 — record boundary node (`i==size-1` / `i==0`)
"minimum depth", "first level that satisfies X"	Group 3 — early return on condition
"connect next pointers at the same level"	Group 4 — wire nodes within a level
"same depth, different parent" (cousins), per-node metadata	Group 5 — track per-node metadata
"maximum width of a level" (counting null gaps)	Group 6 — position indexing

The One Rule — Snapshot Level Size First

```
Always snapshot the level size BEFORE the inner for loop:
    int size = queue.size();
    for (int i = 0; i < size; i++) { ... }

Without the snapshot, newly added children mix with the current level
and i == size−1 / i == 0 checks become meaningless.
```

DFS / Backtracking `dfs.md`

DFS appears in five forms. The pattern determines naming, return type, and where the answer is built.

When to Use — Signal → Pattern

Problem signal phrase	Pattern / template
"height / depth / sum OF the subtree", combine results upward	#1 Process children first, combine at node (post-order)
"root-to-leaf path", "number built along the path"	#2 Process node first, pass state down (pre-order)
"longest/best path that may bend through a node" (spans both arms)	#3 Tree global answer
"count islands / largest region / flood fill" on a grid	#4 Grid DFS
"can finish all", "detect cycle", "valid course order" (directed)	#5 Graph DFS 3-color
"all subsets / permutations / combinations", "find every way"	#6 Backtracking
"valid BST", "kth smallest", "closest value", "successor"	BST templates (pass bounds / inorder / binary search)

All Templates

Variables: `node` = current tree node · `accumulated` / `current` = state carried down a path · `answer` /global = best cross-node result · `grid[r][c]` = cell at row `r`, col `c` · `state[node]` = 0 unseen / 1 in-stack / 2 done · `start` = first eligible index · `current` = in-progress candidate · `nums[i]` = element being chosen **Pseudocode:**

```
if node null: return BASE; left=dfs(left); right=dfs(right); return combine(left,right,node.val)
if node null: return; accumulated+=node.val; if leaf: record; dfs(left,accumulated); dfs(right,accumulate
if node null: return 0; left=max(0,dfs(left)); right=max(0,dfs(right)); answer=max(answer,left+right+node
if out of bounds or not TARGET: return; mark VISITED; recurse into 4 neighbors
if state==1: cycle; if state==2: safe; mark in-stack; recurse neighbors (cycle? return); mark done
if base: record copy; for i from start: skip dups; choose nums[i]; recurse(next); undo
```

```java
// 1. PROCESS CHILDREN FIRST, COMBINE AT NODE (post-order)
// each node trusts its children to return a finished answer, then merges them.  — WHEN: "what is the h
private int dfs(TreeNode node) {
    if (node == null) return BASE;          // 0 for depth, true for balanced
    int left  = dfs(node.left);
    int right = dfs(node.right);
    // combine left, right, node.val → answer propagates up
    return ...;
}

// 2. PROCESS NODE FIRST, PASS STATE DOWN (pre-order)
// carry what you've seen so far down the path; the leaf has the full picture.  — WHEN: "root-to-leaf p
private void dfs(TreeNode node, int accumulated) {
    if (node == null) return;
    accumulated += node.val;                // update state on way down
    if (isLeaf(node)) { /* record */ return; }
    dfs(node.left,  accumulated);
    dfs(node.right, accumulated);
    // backtrack: no undo needed for primitive; undo List.add if using collection
}

// 3. TREE — GLOBAL ANSWER (return up the best single arm; update global across both arms)
// the answer may bend through a node using both arms, but only one arm can extend upward.  — WHEN: use
int answer = Integer.MIN_VALUE;
private int dfs(TreeNode node) {
    if (node == null) return 0;
    int left  = Math.max(0, dfs(node.left));   // clamp negative
    int right = Math.max(0, dfs(node.right));
    answer = Math.max(answer, left + right + node.val);   // cross-node path
    return Math.max(left, right) + node.val;              // single arm up
}

// 4. GRID DFS — mark visited by mutating grid; 4-directional flood fill
// stand on a cell, flood into all 4 neighbors, sinking each as you visit it.  — WHEN: "connected regio
private void dfs(int[][] grid, int r, int c) {
    if (r < 0 || r >= grid.length || c < 0 || c >= grid[0].length) return;
    if (grid[r][c] != TARGET) return;
    grid[r][c] = VISITED;
    dfs(grid, r+1, c); dfs(grid, r-1, c);
    dfs(grid, r, c+1); dfs(grid, r, c-1);
}

// 5. GRAPH DFS — 3-color visited: 0=unseen 1=in-stack 2=done
// if you re-enter a node still on the current stack, you've looped back → cycle.  — WHEN: "can finish
int[] state;
private boolean dfs(int node) {
    if (state[node] == 1) return true;    // back edge → cycle
    if (state[node] == 2) return false;   // already safe
    state[node] = 1;
    for (int neighbor : graph.get(node))
        if (dfs(neighbor)) return true;
    state[node] = 2;
    return false;
}

// 6. BACKTRACKING — choose / recurse / undo
// build a candidate one element at a time; undo each choice to explore the next branch.  — WHEN: "all
private void backtrack(int start, List<Integer> current) {
    if (baseCase) { result.add(new ArrayList<>(current)); return; }
    for (int i = start; i < n; i++) {
        if (skipCondition(i)) continue;    // prune duplicates
        current.add(nums[i]);              // choose
        backtrack(next, current);          // next = i (reuse) or i+1 (no reuse)
```

```
        current.remove(current.size()-1);  // undo
    }
}
```

Graph `graph.md`

`Queue<Integer> queue = new ArrayDeque<>()` throughout.
Six algorithms — each with a canonical template and representative problems.

Complexity Legend

```
BFS / DFS         O(V + E)            visit each vertex and edge once
Topo sort (Kahn)  O(V + E)            each node enqueued once, each edge relaxed once
Union-Find        ~O(n·α(n)) ≈ O(n)   α = inverse Ackermann, effectively constant
Dijkstra          O((V + E) log V)    non-negative weights ONLY
```

Graph Representation

```
// Adjacency list (most common)
List<List<Integer>> graph = new ArrayList<>();
for (int i = 0; i < n; i++) graph.add(new ArrayList<>());
for (int[] edge : edges) {
    graph.get(edge[0]).add(edge[1]);
    graph.get(edge[1]).add(edge[0]);   // omit for directed
}

// Weighted adjacency list: int[] = {neighbor, weight}
List<List<int[]>> graph = new ArrayList<>();
for (int i = 0; i < n; i++) graph.add(new ArrayList<>());
for (int[] edge : edges)
    graph.get(edge[0]).add(new int[]{edge[1], edge[2]});

// Grid 4-directional neighbors
int[] dr = {1, -1, 0,  0};
int[] dc = {0,  0, 1, -1};
```

Core Templates

Variables: `queue` = BFS frontier · `visited[]` = seen flags · `level` = current distance ring · `inDegree[]` = remaining prerequisites per node · `order` = topo result · `parent[]` / `rank[]` = union-find forest · `dist[]` = shortest distance · `pq` = min-heap on cost · `color[]` = 2-coloring (-1/0/1) **Pseudocode:**

```
BFS: mark start visited, enqueue; while queue, snapshot level size, pop each, push unvisited neighbors, i
TOPO: count inDegree per edge, enqueue all zero-inDegree, pop into order and decrement neighbors, enqueue
UNION-FIND: init parent[i]=i; find follows parent to root with path compression; union links roots by ran
DIJKSTRA: dist[src]=0, push src; pop closest, skip if stale, relax each neighbor and push improved
BIPARTITE: color each component start 0, BFS painting neighbors opposite, return false on same-color clas
```

```java
// 1. BFS — shortest path / level order (track distance by level-size snapshot)
// explore in rings of equal distance; snapshot the level size so each ring = one step.  — WHEN: "fewes
Queue<Integer> queue = new ArrayDeque<>();
boolean[] visited = new boolean[n];
queue.offer(start);
visited[start] = true;
int level = 0;
while (!queue.isEmpty()) {
    int size = queue.size();              // ← snapshot: all nodes at this distance
    for (int i = 0; i < size; i++) {
        int node = queue.poll();
        // process node (distance == level)
        for (int neighbor : graph.get(node)) {
            if (!visited[neighbor]) {
                visited[neighbor] = true;
                queue.offer(neighbor);
            }
        }
    }
    level++;
}

// 2. TOPOLOGICAL SORT — Kahn's BFS
// peel off nodes with no remaining prerequisites; if any get stuck, a cycle exists.  — WHEN: "valid or
int[] inDegree = new int[n];
for (int[] edge : edges) inDegree[edge[1]]++;
Queue<Integer> queue = new ArrayDeque<>();
for (int i = 0; i < n; i++) if (inDegree[i] == 0) queue.offer(i);
List<Integer> order = new ArrayList<>();
while (!queue.isEmpty()) {
    int node = queue.poll();
    order.add(node);
    for (int neighbor : graph.get(node))
        if (--inDegree[neighbor] == 0) queue.offer(neighbor);
}
// order.size() == n → no cycle

// 3. UNION FIND
// each set points to one representative; merge sets by linking roots, query by finding roots.  — WHEN:
int[] parent, rank;
void init(int n) {
    parent = new int[n]; rank = new int[n];
    for (int i = 0; i < n; i++) parent[i] = i;
}
int find(int x) {
    if (parent[x] != x) parent[x] = find(parent[x]);  // path compression
    return parent[x];
}
boolean union(int x, int y) {
    int px = find(x), py = find(y);
    if (px == py) return false;
    if (rank[px] < rank[py]) { int t = px; px = py; py = t; }
    parent[py] = px;
    if (rank[px] == rank[py]) rank[px]++;
    return true;
}

// 4. DIJKSTRA — shortest path (non-negative weights)
// greedily settle the closest unfinished node; non-negative weights guarantee it's final.  — WHEN: "sh
int[] dist = new int[n];
Arrays.fill(dist, Integer.MAX_VALUE);
dist[src] = 0;
PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) -> a[1] - b[1]);  // min-heap on cost
```

```java
pq.offer(new int[]{src, 0});
while (!pq.isEmpty()) {
    int[] current = pq.poll();
    int node = current[0], d = current[1];
    if (d > dist[node]) continue;   // stale entry
    for (int[] nb : graph.get(node)) {
        int next = nb[0], w = nb[1];
        if (dist[node] + w < dist[next]) {
            dist[next] = dist[node] + w;
            pq.offer(new int[]{next, dist[next]});
        }
    }
}

// 5. BIPARTITE CHECK — BFS 2-coloring
// paint neighbors the opposite color; a neighbor that's already your color means an odd cycle.  — WHEN
int[] color = new int[n];
Arrays.fill(color, -1);
Queue<Integer> queue = new ArrayDeque<>();
for (int start = 0; start < n; start++) {
    if (color[start] != -1) continue;
    color[start] = 0;
    queue.offer(start);
    while (!queue.isEmpty()) {
        int node = queue.poll();
        for (int neighbor : graph.get(node)) {
            if (color[neighbor] == -1) { color[neighbor] = 1 - color[node]; queue.offer(neighbor); }
            else if (color[neighbor] == color[node]) return false;  // conflict
        }
    }
}
return true;
```

Greedy `greedy.md`

Two interval templates plus non-interval greedy patterns.

Two Interval Templates

```
MERGE DIRECTION RULE (for merge-style problems like #56, where both keys work):
    Sort by START  →  traverse FORWARD  (i = 0 → n-1), extend the END of last kept
    Sort by END    →  traverse BACKWARD (i = n-1 → 0), extend the START of last kept
    These two are exact mirror images. See #56 for both written out side by side.
    (NOTE: this mirror only applies to merging. Template 1 below sorts by end but
     still sweeps FORWARD — there the goal is counting, not merging.)

TEMPLATE 1 — Sort by END time, sweep forward, track end of last kept interval
             Use when: maximizing count of non-overlapping intervals
             Greedy insight: earliest-finishing interval leaves most room for future ones

TEMPLATE 2 — Sort by START time, sweep forward, merge when overlapping
             Use when: merging/counting overlapping intervals
             Greedy insight: processing in start order → each interval only needs to
             compare with the last merged result
```

Variables: `intervals` = list of `[start, end]` · `lastEnd` = end of last kept interval · `result` = merged output · `pq` = min-heap of active end times **Pseudocode:**

```
TEMPLATE 1 (sort by end, count non-overlapping):
  sort intervals by END time
  lastEnd = -infinity
  for each interval:
    if interval.start >= lastEnd:   # no overlap
      lastEnd = interval.end
      keep it (count++ or add)
    else:
      skip it (overlap)

TEMPLATE 2 (sort by start, merge):
  sort intervals by START time
  for each interval:
    if result empty OR last kept end < interval.start:
      add interval as new
    else:
      extend last kept end = max(last end, interval.end)

TEMPLATE 3 (sort by start + min-heap of ends):
  sort intervals by START time
  for each interval:
    if heap nonempty AND earliest end <= interval.start:
      pop heap            # reuse freed resource
    push interval.end     # occupy a resource
  heap size = resources needed
```

```
// sort to expose a local greedy choice, then sweep once making the locally-best pick.  — WHEN: interva
// TEMPLATE 1: Sort by end, compare start
Arrays.sort(intervals, (a, b) -> a[1] - b[1]);      // sort by END time
int lastEnd = Integer.MIN_VALUE;
for (int[] interval : intervals) {
    if (interval[0] >= lastEnd) {                    // start >= lastEnd → no overlap → keep
        lastEnd = interval[1];
        // count++ or add to result
    } else {
        // overlap → skip (or count as removed)
    }
}

// TEMPLATE 2: Sort by start, compare end
Arrays.sort(intervals, (a, b) -> a[0] - b[0]);      // sort by START time
List<int[]> result = new ArrayList<>();
for (int[] interval : intervals) {
    if (result.isEmpty() || result.get(result.size()-1)[1] < interval[0]) {
        result.add(interval);                        // no overlap → new interval
    } else {
        result.get(result.size()-1)[1] =             // overlap → extend end
            Math.max(result.get(result.size()-1)[1], interval[1]);
    }
}

// TEMPLATE 3: Sort by start + min-heap of end times (multi-resource)
// USE WHEN counting simultaneous/overlapping resources — "minimum rooms", "max concurrent".
Arrays.sort(intervals, (a, b) -> a[0] - b[0]);
PriorityQueue<Integer> pq = new PriorityQueue<>();   // min-heap of active end times
for (int[] interval : intervals) {
    if (!pq.isEmpty() && pq.peek() <= interval[0])
        pq.poll();                                   // reuse: earliest-ending resource is free
    pq.offer(interval[1]);                           // occupy a resource until interval[1]
}
// pq.size() = resources needed
```

Dynamic Programming `dynamic_programming.md`

Six template families. Every problem maps to one loop skeleton and one dp-state definition.

All Six Templates

Variables: `dp[i]` = the answer for the subproblem at index/state `i` (meaning varies per family: ways/cost/length/feasibility ending at or reaching `i`) · `n` = problem size · `target` / `j` = knapsack capacity dimension.
Pseudocode:

```
LINEAR 1D:    dp[i] = fixed recipe over dp[i-1], dp[i-2] (Kadane: best ending at i = max(start fresh, exte
LOOK-BACK:    for each i, scan all earlier j and extend the best valid dp[j]
KNAPSACK:     collapse item dimension to 1D; descending j = each item once, ascending j = reusable
2D SEQUENCE: match consumes both strings (diagonal), mismatch drops one side
INTERVAL:     solve short ranges first, combine via a split point inside the range
GRID:         each cell accumulates from cells it could arrive from (up / left)
STATE MACHINE: name a few states, update each from yesterday's states every step
```

```
// 1. LINEAR 1D — each cell depends on a fixed number of previous cells
// the answer at i is a fixed recipe over the last one or two answers.  — WHEN: "ways/cost to reach ste
int[] dp = new int[n + 1];
dp[0] = base;
for (int i = 1; i <= n; i++)
    dp[i] = f(dp[i-1], dp[i-2], ...);
// KADANE (max subarray, #53) = Linear-1D where dp[i] = best sum ENDING at i:
//   dp[i] = max(nums[i], dp[i-1] + nums[i])   // start fresh vs extend (drop a negative prefix)
//   answer = max over all dp[i]; collapse dp[] to one rolling int → O(1) space.
//   variants: #152 product (track max AND min, a negative swaps them); #918 circular.

// 2A. LOOK-BACK 1D — dp[i] depends on all j < i
// to finish position i, scan every earlier position j and extend the best valid one.  — WHEN: "longest
int[] dp = new int[n];
for (int i = 0; i < n; i++) {
    for (int j = 0; j < i; j++) {
        dp[i] = f(dp[i], dp[j]);        // e.g., LIS, word break
    }
}

// 2B/2C. KNAPSACK — collapse the item dimension to 1D; loop direction picks reuse.
//   0/1 (each item once): j DESCENDING  → reads previous row.
//   unbounded (reusable): j ASCENDING   → reads current row.
//   Mnemonic: DESCENDING = Distinct, ASCENDING = Again. Full code + permutation
//   variant + "why" in Part 2 — Knapsack DP.

// 3. 2D SEQUENCE — match consumes both, mismatch drops one side. WHEN: LCS / edit
//    distance / subsequence count over two strings. Full code → Part 3.
// 4. INTERVAL DP — short ranges first (i right-to-left, j from i+1; dp[i+1][*] ready).
//    WHEN: palindrome / merge-burst / split-point in a range. Full code → Part 4.
// 5. GRID DP — each cell accumulates from cells it could arrive from (up/left).
//    WHEN: grid paths / min-cost / largest square. Full code → Part 5.
// 6. STATE MACHINE — named states updated from yesterday's states each step.
//    WHEN: buy/sell/hold, limited transactions, cooldown/fee. Full code → Part 6.
```

Knapsack Decision Tree

```
Want count or min/max?
├── Count (dp[j] += dp[j - num])
│   ├── 0/1 (each item once):    j descending → #416, #494
│   └── Unbounded (reuse):       j ascending  → #518
│       └── Ordered (permuts):   target outer, nums inner → #377
└── Min/Max (dp[j] = min/max(...))
    ├── 0/1 minimize:            j descending → #474 (maximize)
    └── Unbounded minimize:      j ascending  → #322
```

Part 2 — Knapsack DP

Variables: `dp[j]` = number of ways to reach sum `j` (count flavor; `dp[0]=1` is the empty selection); `i` = item index, `j` = current capacity/target, `num` = an item's weight/value. Loop *direction* decides reuse: descending j = item used at most once, ascending j = item reusable; target-outer = orderings counted. **Pseudocode:**

```
0/1 KNAPSACK (each item once):
    dp[0] = 1
    for each item i:
        for j from target DOWN TO nums[i]:     // DESCENDING reads OLD dp (item i not yet used)
            dp[j] += dp[j - nums[i]]

UNBOUNDED KNAPSACK (item reusable):
    dp[0] = 1
    for each item i:
        for j from nums[i] UP TO target:       // ASCENDING reads NEW dp (item i already used this pass)
            dp[j] += dp[j - nums[i]]

PERMUTATION COUNT (order matters):
    dp[0] = 1
    for j from 1 to target:                     // OUTER = target value
        for each num in nums:                   // INNER = try every num, so orderings differ
            if j >= num: dp[j] += dp[j - num]
```

```java
// ── 0/1 KNAPSACK ── each item at most once
int[] dp = new int[target + 1];
dp[0] = 1;
for (int i = 0; i < nums.length; i++) {
    for (int j = target; j >= nums[i]; j--)   // DESCENDING: sees OLD dp (before item i)
        dp[j] += dp[j - nums[i]];
}

// ── UNBOUNDED KNAPSACK ── each item reusable
int[] dp = new int[target + 1];
dp[0] = 1;
for (int i = 0; i < nums.length; i++) {
    for (int j = nums[i]; j <= target; j++)    // ASCENDING: sees NEW dp (after item i already used)
        dp[j] += dp[j - nums[i]];
}

// ── PERMUTATION COUNT ── order matters (Combination Sum IV)
int[] dp = new int[target + 1];
dp[0] = 1;
for (int j = 1; j <= target; j++) {           // OUTER: target
    for (int num : nums) {                     // INNER: try all nums (not fixing order)
        if (j >= num) dp[j] += dp[j - num];
    }
}
```

Why descending for 0/1:
`dp[j - nums[i]]` is read **before** `dp[j]` is updated (j > j-nums[i]). So it always sees the dp state from before item `i` was considered — no item is used twice.

Why ascending for unbounded:
`dp[j - nums[i]]` was already updated in this same pass (j - nums[i] < j, processed earlier). This allows item `i` to be picked multiple times.

2D anchor: both loops are `dp[i][j] = dp[i-1][j] + dp[i-?][j-w]` collapsed to 1D — descending keeps `dp[j-w]` on the *previous* row (item i unused), ascending pulls it from the *current* row (item i reused).

Mnemonic: DESCENDING = **D**istinct (each item once), ASCENDING = **A**gain (reusable).

Flavor = seed + operator (same skeleton):

- count → `dp[0]=1`, `dp[j] += dp[j-w]`

- feasibility → `dp[0]=true`, `dp[j] = dp[j] || dp[j-w]`
- max-value → `dp[0]=0`, `dp[j] = Math.max(dp[j], dp[j-w] + v)`

Part 3 — 2D Sequence DP

Variables: `dp[i][j]` = the answer for the prefixes `s1[0..i)` (first `i` chars) and `s2[0..j)` (first `j` chars); index `0` = empty prefix. **Pseudocode:**

```
dp = (m+1) x (n+1) table
initialize dp[0][*] and dp[*][0] from the empty-prefix base cases
for i from 1 to m:                          // each char of s1
    for j from 1 to n:                      // each char of s2
        if s1[i-1] == s2[j-1]:              // current chars match
            dp[i][j] = dp[i-1][j-1] + 1     // extend the diagonal (both prefixes shrink by 1)
        else:
            dp[i][j] = combine(dp[i-1][j], dp[i][j-1], dp[i-1][j-1])  // drop a char from one/both side
return dp[m][n]
```

```
// i indexes string/array 1 (1-indexed), j indexes string/array 2
int[][] dp = new int[m + 1][n + 1];
// Init dp[0][*] and dp[*][0] based on problem
for (int i = 1; i <= m; i++) {
    for (int j = 1; j <= n; j++) {
        if (s1.charAt(i-1) == s2.charAt(j-1)) {
            dp[i][j] = dp[i-1][j-1] + 1;          // characters match
        } else {
            dp[i][j] = combine(dp[i-1][j], dp[i][j-1], dp[i-1][j-1]);
        }
    }
}
```

Part 4 — Interval DP

Rule: `i` goes right-to-left (ensures shorter/inner intervals are computed first). `j` goes left-to-right from `i+1`. When computing `dp[i][j]`, all `dp[i'][j']` with `i' > i` or `j' < j` are already done.

Variables: `dp[i][j]` = the answer for the interval `[i, j]` (left endpoint `i`, right endpoint `j`). **Pseudocode:**

```
dp = n x n table
for each i: dp[i][i] = base_case            // length-1 intervals
for i from n-1 down to 0:                    // i RIGHT-TO-LEFT so dp[i+1][*] (inner) is ready
    for j from i+1 to n-1:                   // j LEFT-TO-RIGHT from i+1 so dp[i][j-1] is ready
        // safe to read dp[i+1][j], dp[i][j-1], dp[i+1][j-1] (all strictly smaller intervals)
        dp[i][j] = ...transition over interval [i,j]...
```

```
int[][] dp = new int[n][n];
for (int i = 0; i < n; i++) dp[i][i] = base_case;   // single elements
for (int i = n - 1; i >= 0; i--) {                   // right-to-left
    for (int j = i + 1; j < n; j++) {                // left-to-right from i+1
        // dp[i][j] can safely use dp[i+1][j], dp[i][j-1], dp[i+1][j-1]
        dp[i][j] = ...;
    }
}
```

Alternative (forward `i`, inner `j` descending): `i` is the right endpoint going left-to-right; `j` is the left endpoint sweeping back from `i-1`. Inner intervals (larger `j`, smaller `i`) are already filled.

Variables: `dp[i][j]` = the answer for the interval `[j, i]` (right endpoint `i`, left endpoint `j`). **Pseudocode:**

```
dp = n x n table
for each i: dp[i][i] = base_case            // length-1 intervals
for i from 0 to n-1:                         // i = RIGHT endpoint, FORWARD left-to-right
    for j from i-1 down to 0:                // j = LEFT endpoint, DESCENDING from i-1 to 0
        // safe to read dp[i-1][j], dp[i][j+1], dp[i-1][j+1] (all shorter intervals already filled)
        dp[i][j] = ...transition over interval [j,i]...
```

```java
int[][] dp = new int[n][n];
for (int i = 0; i < n; i++) dp[i][i] = base_case;   // single elements
for (int i = 0; i < n; i++) {                         // i = right endpoint, left-to-right
    for (int j = i - 1; j >= 0; j--) {                // j = left endpoint, from i-1 down to 0
        // dp[i][j] can safely use dp[i-1][j], dp[i][j+1], dp[i-1][j+1]
        dp[i][j] = ...;
    }
}
```

Alternative (triangular fill, column base case): used when `dp[i][j]` depends only on the previous row/column (e.g. counting problems). Fill row by row with `j` from `0` to `i`.

Variables: `dp[i][j]` = the count/answer for the `(i, j)` subproblem, defined only for `j <= i` (lower-triangular). **Pseudocode:**

```
dp = n x n table
for each i: dp[i][0] = 1                     // column base case; dp[0][j>0] = 0
for i from 1 to n-1:                         // fill ROW BY ROW, top to bottom
    for j from 1 to i:                       // j from 1 up to i (triangular, j <= i)
        // safe to read dp[i-1][j], dp[i][j-1], dp[i-1][j-1] (prior row / earlier in row)
        dp[i][j] = ...transition...
```

```java
int[][] dp = new int[n][n];
for (int i = 0; i < n; i++) dp[i][0] = 1;   // base: dp[i][0] = 1; dp[0][j] = 0 for j > 0
for (int i = 1; i < n; i++) {
    for (int j = 1; j <= i; j++) {           // j from 1 to i
        // dp[i][j] can safely use dp[i-1][j], dp[i][j-1], dp[i-1][j-1]
        dp[i][j] = ...;
    }
}
```

Part 5 — Grid DP

Variables: `dp[i][j]` = best (path count / cost) to reach cell `(i, j)` moving only right/down; `memo[i][j]` = longest path starting from `(i, j)` for the all-direction DFS variant; `dr` / `dc` = 4-direction deltas. **Pseudocode:**

```
dr = {1,-1,0,0}; dc = {0,0,1,-1}            // DOWN UP RIGHT LEFT

// Standard grid (only right/down — DAG, no cycle):
dp = rows x cols table
initialize first row and first column
for i from 0 to rows-1:
    for j from 0 to cols-1:
        dp[i][j] = grid[i][j] + f(dp[i-1][j], dp[i][j-1])   // combine top & left

// All-direction grid (memoized DFS for increasing/decreasing paths):
memo = rows x cols table
for i from 0 to rows-1:
    for j from 0 to cols-1:
        dfs(grid, i, j, memo, dr, dc)        // each cell caches its longest path
```

```java
int[] dr = {1,-1,0,0}, dc = {0,0,1,-1};   // DOWN UP RIGHT LEFT

// Standard grid (only move right/down — DAG, no cycle):
int[][] dp = new int[rows][cols];
// initialize first row and first column
for (int i = 0; i < rows; i++)
    for (int j = 0; j < cols; j++)
        dp[i][j] = grid[i][j] + f(dp[i-1][j], dp[i][j-1]);

// All-direction grid (memoized DFS for increasing/decreasing paths):
int[][] memo = new int[rows][cols];
for (int i = 0; i < rows; i++)
    for (int j = 0; j < cols; j++)
        dfs(grid, i, j, memo, dr, dc);
```

Part 6 — State Machine DP (Stock Problems)

```
cash      = max profit when NOT holding (free to buy or idle)
hold      = max profit when HOLDING stock (bought it at some cost)
cooldown  = max profit right after selling (can't buy today)
```

Part 6 — State Machine DP (Stock Problems)

```
                    buy                 sell                re-buy condition
#121 (1 tx):        hold = max(hold, -price)                no re-buy (ignore cash)
#122 (unlimited):   hold = max(hold, cash - price)          re-buy with full cash
#123 (2 tx):        chain: buy2 uses sell1 cash             4 states chained
#309 (cooldown):    hold = max(hold, cooldown - price)      must wait 1 day
#714 (fee):         hold = max(hold, cash - price)          pay fee on sell
```

Bit Manipulation `bit_manipulation.md`

Canonical Template

Variables: `result` = XOR accumulator (lone survivor) · `n` = number being inspected/edited · `i` = bit position · `count` = set-bit counter · `mask` = 26-bit letter-presence set · masks: `1 << i` selects bit i, `n & (n-1)` clears lowest set bit, `n &`

`(-n)` isolates lowest set bit **Pseudocode:**

```
XOR CANCEL:
    result = 0
    for each num in nums: result = result XOR num   (pairs cancel, lone survives)

BIT OPERATIONS on n at position i:
    isSet  = bit i of n is 1            -> (n >> i) & 1 == 1
    set    = turn bit i on             -> n | (1 << i)
    clear  = turn bit i off            -> n & ~(1 << i)
    toggle = flip bit i                -> n ^ (1 << i)
    lowest = isolate lowest set bit    -> n & (-n)
    isPow2 = n > 0 and clearing lowest set bit gives 0

COUNT SET BITS (Brian Kernighan):
    count = 0
    while n != 0: count = count + 1; n = n & (n-1)   (each step removes one set bit)

BITMASK AS SET:
    mask = 0
    for each char c in word: mask = mask | (1 << (c - 'a'))
    two words share no letter when (mask[i] & mask[j]) == 0
```

```java
// bits are a set/counter — XOR cancels duplicates, AND/OR/shift edit individual bits.  — WHEN: "appear
// —— XOR CANCEL —— pairs cancel; lone element survives
int result = 0;
for (int num : nums) result ^= num;

// —— BIT OPERATIONS ——
boolean isSet  = ((n >> i) & 1) == 1;     // check if bit i is set
int set        = n | (1 << i);             // set bit i
int clear      = n & ~(1 << i);            // clear bit i
int toggle     = n ^ (1 << i);             // toggle bit i
int lowest     = n & (-n);                 // isolate lowest set bit
boolean isPow2 = n > 0 && (n & (n-1)) == 0;

// —— COUNT SET BITS —— Brian Kernighan: each iteration removes one set bit
// WHY n & (n-1) clears the lowest set bit:
//   n      = ...1000   (lowest set bit at this position)
//   n-1    = ...0111   (borrow flips that bit to 0 and all lower bits to 1)
//   n&(n-1)=...0000   (AND keeps higher bits, wipes the lowest set bit + below)
int count = 0;
while (n != 0) { count++; n &= (n - 1); }  // n & (n-1) clears lowest set bit

// —— BITMASK AS SET —— 26-bit integer represents presence of 26 letters
int mask = 0;
for (char c : word.toCharArray()) mask |= (1 << (c - 'a'));
// check no overlap between two words:
if ((mask[i] & mask[j]) == 0) { /* no shared letter */ }
```

Variations

Variables: `ones` / `twos` = bits seen 1 / 2 times mod 3 · `xor` = `a ^ b` of the two uniques · `diff` = lowest differing bit (`xor & -xor`) · `a` / `b` = the two unique values · `dp[]` = set-bit counts by number · `shift` = bits dropped to reach common prefix · `carry` = AND-derived carry bits **Pseudocode:**

```
VARIATION 1 (mod-3 state machine, Single Number II):
    ones = 0; twos = 0
    for each num: ones = (ones XOR num) AND NOT twos; twos = (twos XOR num) AND NOT ones
    after loop, ones holds the element appearing once

VARIATION 2 (split XOR into two groups, Single Number III):
    xor = XOR of all nums                 (= a XOR b)
    diff = xor AND (-xor)                 (lowest bit where a and b differ)
    a = 0; for each num: if (num AND diff) != 0: a = a XOR num   (XOR one group)
    b = xor XOR a                         (the other unique)

VARIATION 3 (DP relation, Counting Bits):
    dp = array of size n+1
    for i = 1..n: dp[i] = dp[i >> 1] + (i AND 1)   (drop last bit, add it back)

VARIATION 4 (common prefix, Bitwise AND of Range):
    shift = 0
    while left != right: left >>= 1; right >>= 1; shift = shift + 1
    return left << shift                  (shared high-bit prefix restored)

VARIATION 5 (carry loop, Sum of Two Integers):
    while b != 0:
        carry = a AND b                   (positions that carry)
        a = a XOR b                       (sum without carry)
        b = carry << 1                    (carry moves one bit left)
    return a
```

```
// VARIATION 1: mod-3 state machine (Single Number II)
// MENTAL MODEL: extend XOR from mod-2 (cancel pairs) to mod-3 (cancel triples);
//               ones/twos track bits seen 1 or 2 times mod 3.
// Instead of XOR canceling pairs (mod 2), cancel triples (mod 3)
// ones = bits seen exactly 1 mod 3 times; twos = bits seen exactly 2 mod 3 times
int ones = 0, twos = 0;
for (int num : nums) {
    ones = (ones ^ num) & ~twos;    // ← add to ones if not already in twos
    twos = (twos ^ num) & ~ones;    // ← add to twos if not already in ones
}
// After all: ones holds the element appearing once (0 mod 3 remainder = survive)

// VARIATION 2: split XOR into two groups (Single Number III)
// Two unique elements a, b. XOR of all = a ^ b.
// Use lowest set bit of (a^b) to split nums into two groups → each group has one unique.
int xor  = 0;
for (int num : nums) xor ^= num;      // xor = a ^ b
int diff = xor & (-xor);              // ← lowest set bit that differs between a and b
int a = 0;
for (int num : nums)
    if ((num & diff) != 0) a ^= num;  // ← XOR one group → one unique element
int b = xor ^ a;                       // ← the other unique element

// VARIATION 3: DP relation (Counting Bits)
// dp[i] = dp[i/2] + last bit of i
// i >> 1 = i without last bit (same or fewer ones); last bit = i & 1
int[] dp = new int[n + 1];
for (int i = 1; i <= n; i++)
    dp[i] = dp[i >> 1] + (i & 1);    // ← right-shift halves the problem

// VARIATION 4: common prefix (Bitwise AND of Range)
// AND of any range [left, right]: bits that differ between left and right get cleared.
// Right-shift both until equal → find shared high-bit prefix → shift back.
int shift = 0;
while (left != right) { left >>= 1; right >>= 1; shift++; }
return left << shift;                  // ← shift back to original magnitude

// VARIATION 5: carry loop (Sum of Two Integers)
// XOR gives bit sum without carry; AND shifted left gives carry.
// Repeat until no carry remains.
int add(int a, int b) {
    while (b != 0) {
        int carry = a & b;            // ← carry bits
        a = a ^ b;                    // ← partial sum (no carry)
        b = carry << 1;               // ← carry propagated one position left
    }
    return a;
}
```

Design `design.md`

Canonical Template — HashMap + Doubly Linked List (LRU)

Variables: `map` = key -> CacheNode · `head` / `tail` = sentinels (head side = MRU, tail side = LRU) · `node.previous` / `node.next` = doubly linked neighbors **Pseudocode:**

```
CacheNode: key, value, previous, next
init: head <-> tail (head.next = tail; tail.previous = head)

remove(node):
  node.previous.next = node.next
  node.next.previous = node.previous

insertAfterHead(node):          # mark as most-recently-used
  link node between head and head.next

evict LRU:
  lru = tail.previous           # node just before tail
  remove(lru); map.remove(lru.key)
```

```
// HashMap gives O(1) lookup; the doubly linked list orders by recency so the LRU is always one hop fro

// CacheNode for doubly linked list (named CacheNode to distinguish from ListNode contexts)
class CacheNode {
    int key, value;
    CacheNode previous, next;
    CacheNode(int key, int value) { this.key = key; this.value = value; }
}

// ── SENTINEL NODES ── head (MRU side) ↔ ... ↔ tail (LRU side)
CacheNode head = new CacheNode(0, 0), tail = new CacheNode(0, 0);
// Initialize: head <-> tail
head.next = tail; tail.previous = head;

// ── REMOVE NODE ──
void remove(CacheNode node) {
    node.previous.next = node.next;
    node.next.previous = node.previous;
}

// ── INSERT AFTER HEAD (= mark as MRU) ──
void insertAfterHead(CacheNode node) {
    node.next = head.next;
    node.previous = head;
    head.next.previous = node;
    head.next = node;
}

// ── LRU EVICTION ── tail.previous is LRU
CacheNode lru = tail.previous;
remove(lru);
map.remove(lru.key);
```

Variations

Variables: `keyToVal` / `keyToFreq` = key lookups · `freqToKeys` = freq -> ordered keys at that freq · `minFreq` = smallest live frequency · (RandomizedSet) `list` = values array · `map` = val -> index **Pseudocode:**

```
VARIATION 1 (LFU): maps key->value, key->freq, freq->LinkedHashSet<key>, plus minFreq
  on access: freq++; move key from freqToKeys[old] to freqToKeys[new]
  on evict: remove first key (oldest) from freqToKeys[minFreq]

VARIATION 2 (RandomizedSet): ArrayList of values + HashMap val->index
  remove: swap target with last element, update map, drop last
  getRandom: list[random index]
```

```
// VARIATION 1: LFU — adds a frequency dimension
// Need: key→value, key→freq, freq→orderedSet<key>, and minFreq
// When a key is accessed: freq++, move from freqToKeys[oldFreq] to freqToKeys[newFreq]
// On eviction: remove first element from freqToKeys[minFreq]
// LinkedHashSet preserves insertion order → oldest at front for same frequency

// VARIATION 2: Insert Delete GetRandom — use array for O(1) random
// ArrayList stores values; HashMap stores val→index in array
// On remove: swap target with last element, update map, remove last
// On getRandom: return list.get(random.nextInt(list.size()))
```

Math `math.md`

Canonical Templates

Variables: `n` = the number being processed · `digit` = last digit peeled · `x` = base in fast power · `result` = running product/accumulator · `b` = the base for conversion · `composite[]` = sieve marks **Pseudocode:**

```
DIGIT EXTRACTION:
  while n is not zero:
    digit = remainder of n divided by 10
    drop the last digit of n
    use digit

FAST POWER (x^n):
  result starts at 1
  while exponent n still has bits:
    if the lowest bit is set, fold the current x into result
    square x to reach the next power
    shift n right to consume that bit
  return result

BASE CONVERSION (number -> digits):
  while num is positive:
    append num mod b
    divide num by b
  reverse the collected digits
BASE CONVERSION (digits -> number):
  value starts at 0
  for each digit: value = value * b + digit

SIEVE:
  make a composite[] flag array up to n
  for i from 2 while i*i < n:
    if i already marked composite, skip it
    mark every multiple of i starting at i*i as composite
```

```java
// MENTAL MODEL: number = sequence of digits in some base; iterate by repeatedly
//               peeling the last digit (% base) and shifting (/ base).
// WHEN: "reverse digits", "x^n", "base conversion", "count primes", "nth divisible-by".

// ── DIGIT EXTRACTION LOOP ── peel digits right-to-left
while (n != 0) {
    int digit = n % 10;   // last digit
    n /= 10;              // drop last digit
    // ... use digit ...
}

// ── FAST POWER (exponentiation by squaring) ──
// WHY: x^n needs only O(log n) multiplications by squaring the base
//      and consuming one exponent bit per step.
//   x^13 = x^8 * x^4 * x^1   (13 = 1101 in binary)
double fastPow(double x, long n) {
    double result = 1;
    while (n > 0) {
        if ((n & 1) == 1) result *= x;   // bit set → include this power of x
        x *= x;                          // square the base for the next bit
        n >>= 1;                         // consume one exponent bit
    }
    return result;
}

// ── BASE CONVERSION ── generic base b, 0-indexed
// number → digits: peel with % b, shift with / b, collect, then reverse
StringBuilder builder = new StringBuilder();
while (num > 0) {
    builder.append(num % b);
    num /= b;
}
builder.reverse();
// digits → number: Horner's rule
int value = 0;
for (int digit : digits) value = value * b + digit;

// ── SIEVE OF ERATOSTHENES ── mark composites up to n
// WHY start at i*i: any multiple k*i with k < i was already marked by k.
boolean[] composite = new boolean[n];
for (int i = 2; i * i < n; i++) {
    if (composite[i]) continue;
    for (int j = i * i; j < n; j += i)   // start at i*i, step by i
        composite[j] = true;
}
```